C# Session 10: Detailed Explanations

Here's a comprehensive overview of the topics covered in your session, including **Anonymous Types**, **Extension Methods**, **Partial Classes**, **Partial Methods**, **LINQ to Objects**, **Writing LINQ Queries**, **Deferred Execution**, **LINQ Methods**, and **PLINQ**.

---

1. C# Anonymous Types

Overview

- **Definition**: Anonymous types allow you to create objects without explicitly defining a class. They are useful for encapsulating a set of read-only properties.
- **Characteristics**:
    - Created using the `new` keyword with an object initializer.
    - Properties are read-only.
    - Type name is generated by the compiler.
    - Scope limited to the method where defined.

Example

```
csharp
VerifyOpen In EditorRunCopy code
1var person = new { Age = 34, Name = "John" };
2Console.WriteLine($"Name: {person.Name}, Age: {person.Age}");
```

Key Features

- **Read-Only Properties**: Cannot modify properties after creation.
- **Local Scope**: Accessible only within the method.
- **No Methods or Events**: Only properties are allowed.

---

2. C# Extension Methods

Overview

- **Definition**: Extension methods allow you to add new methods to existing types without modifying their source code.
- **Characteristics**:
    - Defined as static methods in a static class.
    - The first parameter specifies the type being extended, prefixed with `this`.

Example

```
csharp
VerifyOpen In EditorRunCopy code
1public static class StringExtensions
2{
3    public static int WordCount(this string str)
4    {
5        return string.IsNullOrEmpty(str) ? 0 : str.Split(' ').Length;
6    }
7}
```

```
8
9// Usage
10string text = "Hello World";
11int count = text.WordCount(); // count = 2
```

Common Use Cases

- Enhancing built-in types.
- Adding functionality to third-party libraries.

---

3. C# Partial Classes

Overview

- **Definition**: Partial classes allow you to split the definition of a class across multiple files.
- **Characteristics**:
  - Each part must be marked with the `partial` keyword.
  - All parts must have the same accessibility modifier.

Example File 1: `Person.Part1.cs`

```
csharp
VerifyOpen In EditorRunCopy code
1public partial class Person
2{
3    public string FirstName { get; set; }
4}
```

File 2: `Person.Part2.cs`

```
csharp
VerifyOpen In EditorRunCopy code
1public partial class Person
2{
3    public string LastName { get; set; }
4}
```

Usage

```
csharp
VerifyOpen In EditorRunCopy code
1var person = new Person { FirstName = "John", LastName = "Doe" };
```

---

4. C# Partial Methods

Overview

- **Definition**: Partial methods allow you to declare a method in one part of a partial class and optionally implement it in another part.
- **Characteristics**:
  - Must be declared with the `partial` keyword.
  - Cannot have access modifiers.

Example File 1: `Car.Part1.cs`

```
csharp
VerifyOpen In EditorRunCopy code
1public partial class Car
2{
3    partial void Initialize();
4}
```

File 2: `Car.Part2.cs`

```
csharp
VerifyOpen In EditorRunCopy code
1public partial class Car
2{
3    partial void Initialize()
4    {
5        Console.WriteLine("Car initialized.");
6    }
7}
```

---

5. C# LINQ to Objects

Overview

- **Definition**: LINQ (Language Integrated Query) allows querying collections in a more readable way.
- **Key Concepts**:
    - **Query Syntax**: Similar to SQL.
    - **Method Syntax**: Uses extension methods.
    - **Deferred Execution**: Queries are executed when iterated.

Example

```
csharp
VerifyOpen In EditorRunCopy code
1var students = new List<Student>
2{
3    new Student { Name = "John", Age = 20 },
4    new Student { Name = "Alice", Age = 17 }
5};
6
7var query = from student in students
8            where student.Age > 18
9            select student.Name;
10
11foreach (var name in query)
12{
13    Console.WriteLine(name);
14}
```

---

6. Writing LINQ Queries

Common LINQ Methods

- **Where**: Filters a collection.
- **Select**: Projects each element into a new form.

- **OrderBy**: Sorts elements.

Example of Grouping

```
csharp
VerifyOpen In EditorRunCopy code
1var grouped = from student in students
2              group student by student.Age into ageGroup
3              select new { Age = ageGroup.Key, Students = ageGroup };
4
5foreach (var group in grouped)
6{
7    Console.WriteLine($"Age Group: {group.Age}");
8}
```

---

7. Deferred Execution

- **Definition**: LINQ queries are not executed until the results are enumerated (e.g., using `foreach`).
- **Implication**: This allows for dynamic queries based on conditions.

Example

```
csharp
VerifyOpen In EditorRunCopy code
1var filteredStudents = students.Where(s => s.Age > 18);
```

The query is executed only when `filteredStudents` is iterated.

---

8. LINQ Methods

- **Aggregate Functions**: `Count()`, `Sum()`, `Average()`, etc.
- **Example**:

```
csharp
VerifyOpen In EditorRunCopy code
1var totalStudents = students.Count();
2var averageAge = students.Average(s => s.Age);
```

---

9. PLINQ (Parallel LINQ)

- **Overview**: PLINQ allows for parallel processing of LINQ queries, improving performance on multi-core processors.
- **Usage**: Simply add `.AsParallel()` to your LINQ query.

Example

```
csharp
VerifyOpen In EditorRunCopy code
1var parallelQuery = students.AsParallel()
2                            .Where(s => s.Age > 18)
3                            .Select(s => s.Name);
```

Conclusion

This session covered essential C# features that enhance code organization, readability, and performance. Understanding these concepts will significantly improve your ability to write efficient and maintainable code. If you have any specific questions or need further clarification on any topic, feel free to ask!